Declared but Not Sampled: Measuring What a Manipulation Benchmark Score Can Certify

Anonymous submission

Abstract

A manipulation benchmark's score can be made of *location* rather than *looking*. We give a defect that causes it, a criterion for when the defect is exploitable, and an honest account of failing to demonstrate the exploit.

The defect. Every LIBERO task declares, in its own task file, a region from which to sample its target's starting position. Three of the four suites ship evaluation states that exercise that region.

LIBERO-Object does not. It declares the largest region of the four, 5×5 cm, and ships the smallest spread: on six of its ten tasks all fifty released states place the target at a single point, to the last bit of a `float64`. This is arithmetic on two shipped files. No tolerance, policy, simulator or statistic enters it, and LIBERO-Long, given the identical region by the same generator, exercises 96.7–99.8% of it.

The criterion. A frozen placement matters only if the robot cannot resolve the freezing. Define the *placement radius* R of an object as the radius of the smallest disc containing every position the benchmark ships for it. A single task-indexed constant serves that object *if and only if* $R \leq \delta$, the tolerance below which the embodiment cannot tell two placements apart.

Where that holds, no function of episode outcomes separates a lookup table from a policy that perceives accurately. On the object its tasks are *named* by, LIBERO-Object satisfies this on all ten tasks at the tolerance we measure, against two of thirty elsewhere — both fixtures bolted to the scene. How weakly δ is determined is reported in full: it is the weakest link, and our two estimates of it disagree.

The failed demonstration. We built the lookup policy, ran it on all ten tasks, and it does not settle the question.

It scores 16/100 where our released policy scores 8/100 — but eight of ten tasks are 0–0 for both, and three tasks handed a constant 0.469 mm from a fourth's, in all three axes, score 0/10 where it scores 10/10. Half a millimetre of goal cannot separate those outcomes: the successes are governed by the object, not the constant. We withdraw the behavioural claim, report the experiment that would replace it, and record the four ways this work misled us — including one error found inside our correction of another.

	probes from	cannot say
Benchmark degeneracy, outside robotics		
Torralba & Efros [12]	dataset identity	—
HANS [8], artifacts [4]	hypothesis only	where in a stack
ImageNet-v2 [10]	a second test set	—
Shortcuts [3]	general statement	—
Perturbation, in robotics		
LIBERO-PRO [13]	re-running with perturbed scenes	how much was gameable *a priori*
LIBERO-Plus [2]	seven axes, ten VLAs	which stage reads the shortcut
robomimic [7], COLOSSEUM [9]	re-collection, perturbation	what the shipped files already permit
This paper	**shipped files, before training**	**whether the radius binds: that needs δ, and δ needs a policy**

Table 1: Every prior entry needs a trained model and a run to say anything. The placement radius is computed from the benchmark's own initial-state files with no policy, no simulator and no training, which is what lets it bound what a score *could* certify rather than report what one model did.

1 The question

A policy scores 0.700 on a manipulation benchmark — 35 successes in 50 trials on one LIBERO-Object task, and it is our own policy. What has been certified?

The intended reading is a capability claim: the policy perceives the named object, locates it, and manipulates it. The reading this paper is about is narrower and much cheaper: the policy has learned *where the object always is*. Both readings produce the same number. Distinguishing them is not a matter of looking harder at the policy — two policies can be indistinguishable to a probe and opposite in the world, and we exhibit such a pair — but of first measuring the benchmark.

The framing is not new either. While this work was in revision, Jiang et al. [5] defined *shortcut solvability*: a benchmark score is evidence of capability only if every policy reaching it has the capability.

They audit five manipulation benchmarks including all of LIBERO, and show a 0.09B probe with no language encoder scoring within one point of the best published result on three of LIBERO's four splits, and 100.0% on LIBERO-Object. That is our Corollary 1 in content and a stronger empirical demonstration of it than ours. Our *blind* arm is likewise the manipulation instance of the unimodal ablation that Thomason et al. [11] established in embodied AI. We claim no novelty for either move.

What remains ours is the *a priori* half. Their diagnostic requires training a probe and running it. R is a property of the shipped files, so it bounds what a score could certify before a GPU is switched on, and it says *which* object in a task carries the degeneracy.

Their result is the symptom. The radius is a candidate explanation for *part* of it and cannot be the whole: on Spatial, Goal and Long our own Table 4 shows large radii, so pinning cannot be why a probe succeeds there. On those splits the route is visual task-identifiability without language, which §7.1 independently supports — so the two findings converge rather than compete. Table 1 places the rest. Measuring a benchmark's degeneracy before crediting a model is standard outside robotics, and our *blind* arm is the manipulation instance of the hypothesis-only baseline; we claim no novelty for the move. That VLAs ignore language is likewise not our finding.

What that literature lacks, and what manipulation makes possible, is a metric *computable from a benchmark's shipped files before any model is trained.* The prior here is not a class frequency but a physical coordinate, and a physical coordinate can be measured exactly.

What those results leave open is the part a practitioner needs. They show *that* scores collapse under perturbation. They do not say how much a given benchmark could have been gamed in principle, which stage of a stack is reading the shortcut, whether the shortcut can be removed, or what evidence would count as showing that it had been. This paper answers those four questions for one suite and one stack, in that order, and the first answer is the one that gives the others their meaning.

Contributions.

1. **A benchmark defect, measurable from shipped files** (§2). LIBERO-Object's released evaluation states do not sample the region its own task files declare; three sibling suites do. No tolerance, policy or simulator is required to check this, and the fix is already specified by the benchmark.
2. **A criterion for when such a defect is exploitable** (§3). The placement radius R, with the exact condition $R \leq \delta$ — necessary and sufficient, not merely sufficient — and a measurement of all four LIBERO suites against it.
3. **A behavioural test that fails, with its diagnosis** (§5–§6). We built the lookup policy and it cannot decide the question; we say why, at the correct unit of analysis, and name the experiment that would.
4. **Certification methodology** (§9): a counterexample to probe-only certification, and the demonstration that a behavioural certificate is joint in *(head, stack)*.
5. **A falsification record** (§11): every prediction we registered including the failures, every instrument retired at its calibration gate, and four errors of our own — one of them found inside the correction of another.

What is regenerable, and what is not. `scripts/verify_paper_numbers.py` recomputes 58 values across §3 and §6 from the per-trial outcomes and the shipped initial states, including the tests those sections turn on — and fails on disagreement.

An earlier version of that script checked p-values by reading them back out of the JSON that wrote them — a consistency check between two copies of a number, not a re-derivation. A reviewer caught it; it is fixed. The script names the claims it does *not* cover, and one of them matters: the historical $0.700 = 35/50$ cell that opens this paper was measured before we shipped per-trial logs, and its trial record is not in the repository.

Scope, stated once and up front. All results are simulation. Every confirmatory cell is one task and one object unless stated. Cell sizes are given with every number and no cell is quoted without its interval. Our external validation attempt is reported as a *negative* (§10): we could not reproduce a public checkpoint's published performance on our build, and we do not report probe results on a harness whose baseline we cannot reproduce.

2 A benchmark that does not sample what it declares

We begin with the fact that needs the least machinery. It compares two files the benchmark ships. No tolerance, no controller, no policy, no simulator, and no statistics enter it.

Each LIBERO task declares, in its own BDDL, an axis-aligned region to sample its target from. For `pick_up_the_alphabet_soup` that region is 5×5 cm. The fifty shipped initial states place the soup at a single point — the box's exact centre (Figure 3). **The generator randomises; the released evaluation states do not.** Figure 2 shows this is not one chosen task: it draws all twenty tasks of the two suites that declare the same box.

So LIBERO-Object's pinning is not a design decision to be argued with. It is a **state-generation defect**: the benchmark's own task files specify the fix, and a suite in the same release demonstrates it working.

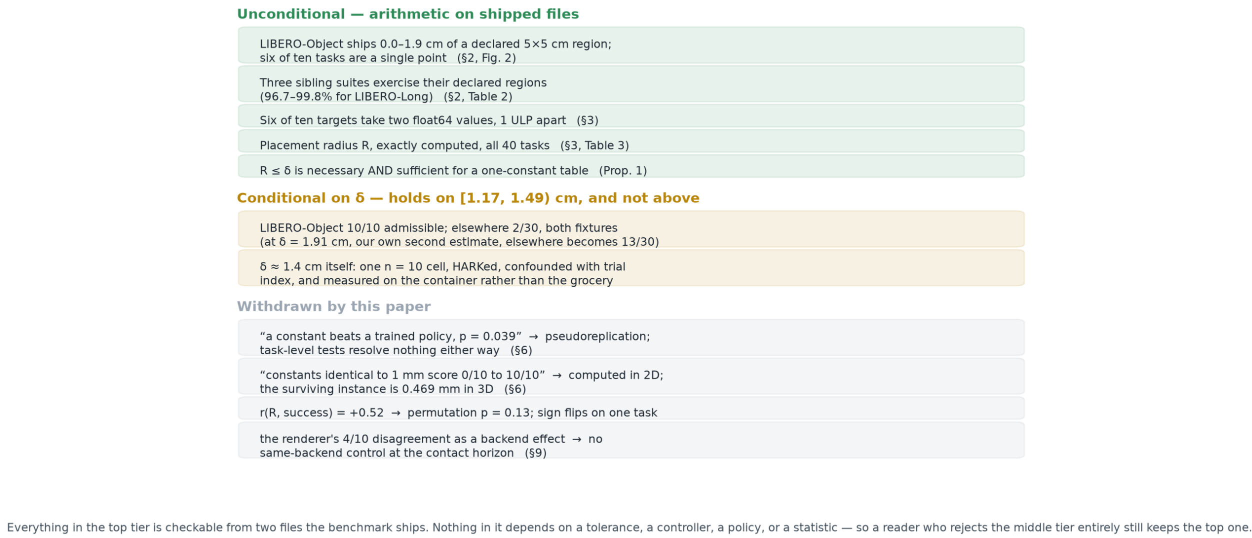

Figure 1: **What each claim in this paper rests on.** The tolerance δ is this work's weakest link — our two measurements of it disagree by a factor of 1.4, and the separation it licenses lives on a 0.32 cm window — so rather than leave a reader to reconstruct the dependency from prose, we draw it. **Nothing in the top tier depends on δ, on a controller, on a policy, or on a statistic**: it is arithmetic on files the benchmark ships. A reader who rejects the middle tier outright still keeps the top one. The bottom tier is claims this paper made and withdrew, kept visible so the record is legible. Generated by `paper/render_fig12_dependency.py`.

suite	declared	shipped spread	exercised?
Spatial	2.0 cm	2.9 cm	yes
Goal	2.0 cm	3.0 cm	yes
Long	5.0 cm	4.9 cm	yes
Object	**5.0** cm	**0.0**–**1.9** cm	**no**

Table 2: **Three LIBERO suites ship states that exercise the region their task files declare. One does not.** Mean declared box side against mean shipped spread. LIBERO's sampler places within the declared box plus a margin, so a suite doing its job ships a spread at or slightly *above* its declaration — Spatial, Goal and Long all do. LIBERO-Object declares the largest box of the four and ships the smallest spread, 0.0 cm on six of ten tasks, where all fifty states are one point. Long is the sharpest control: it declares the *identical* 5 cm box and ships 96.7–99.8% of it, so this is not a property of the generator. Arithmetic on two shipped files — no δ, no policy, no simulator. Artifact: `results/declared_vs_shipped.json`.

We ship the repair, not a recommendation. The fix requires no redesign: draw the target's position from the box the task file already declares. `scripts/resample_init_states.py` does this for all ten LIBERO-Object tasks, changing *only* the two state-array columns holding the target's x and y — every other object, every joint velocity and the time field are bit-identical to the released file, which we verify by diffing them.

That leaves the question this paper exists to answer. A frozen placement is only a problem if a policy can exploit it, and whether it can is a question about the *embodiment* as much as the files. §3 makes that precise, and §6 reports our attempt to demonstrate it, which failed.

	shipped	repaired
spread, LIBERO-Object (10 tasks)	0.00–1.94 cm	4.83–4.97 cm
spread, LIBERO-Long (unmodified)	4.84–4.99 cm	

Table 3: The repaired LIBERO-Object states span the region their task files declare, and land on the same spread as the sibling suite that never needed repairing. The candidate suite is in `results/resampled_init`. We state its limit plainly: positions are drawn from the region the benchmark itself declares valid, so they should be admissible by construction, but we have not run them through a simulator to confirm no interpenetration. It is a candidate repair, and the manifest says so.

How we pick the identity-bearing object, and how much it matters. We take it to be the first entry of the task's declared `obj_of_interest`. That is *not* the same as "the object unique to this task": in 22 of 40 tasks the first entry is shared with a sibling task, and LIBERO-Object is the one suite where the two rules coincide.

Under the stated semantics instead — take the entry unique to the task — the count elsewhere goes from 2/30 to **8/30**; taking the last entry (the destination) reverses the result entirely, to 0/10 against 16/30. The verdict is a function of a labelling choice, and we report all three (`results/admissibility.json`).

The separation is partly built into how LIBERO is organised, and we should say so. A referee put this sharply and it is correct. LIBERO-Object's ten tasks are individuated by *which object*, so the suite has no need

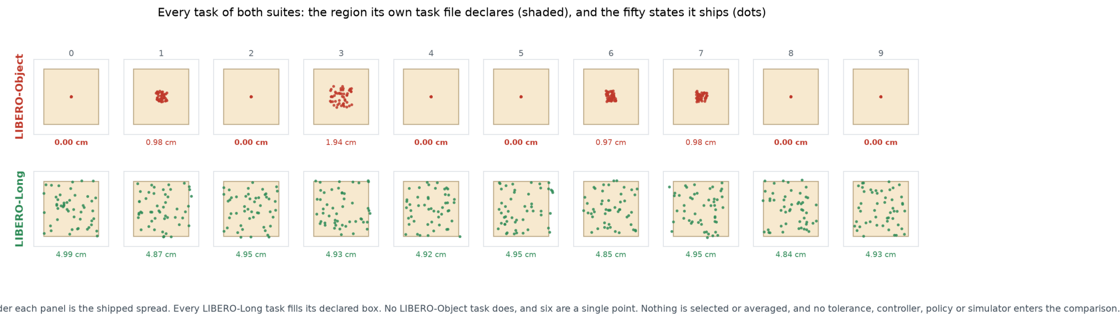

Figure 2: **Every task of both suites — nothing selected, nothing averaged.** Each panel is one task on identical axes: the shaded square is the sampling region that task's own BDDL declares, the dots are the fifty initial states the benchmark ships, and the number beneath is the shipped spread. **Every LIBERO-Long task fills its declared box; no LIBERO-Object task does, and six are a single point.** The two rows are the whole of §2, and no tolerance, controller, policy or simulator enters the comparison. Generated by `paper/render_fig13_allsmall.py`.

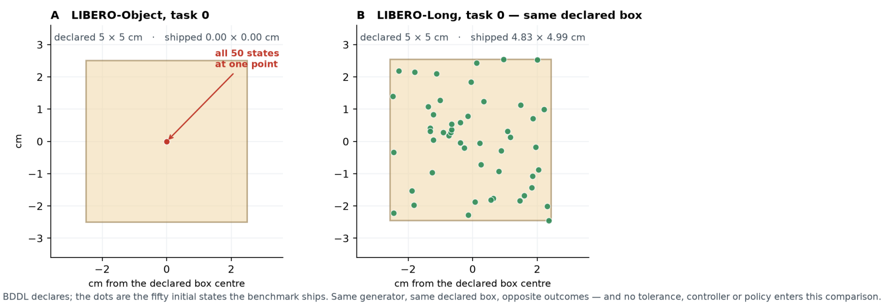

Figure 3: **The region a LIBERO task declares, against the states it ships.** The shaded square is the sampling box the task's own BDDL specifies; the dots are the fifty shipped initial states. **A** LIBERO-Object task 0: a 5×5 cm declaration, and all fifty states at one point. **B** LIBERO-Long task 0, same generator, the identical 5×5 cm declaration, drawn on the same scale: the states fill it. No tolerance, controller, policy or simulator enters this comparison — it is two shipped files. Generated by `paper/render_fig10_declared.py`.

to move it. LIBERO-Spatial's are individuated by *where the bowl is*, so its identity-bearing quantity *must* vary or the suite would not exist. To that extent Table 4's contrast restates the four suites' design intents rather than discovering a defect in them.

What is not built in is the magnitude. Nothing about "individuate by object" requires the position to be frozen to one `float64` value when the task file declares a 5×5 cm region to sample from; LIBERO-Object could randomise placement freely and still individuate by identity, and LIBERO-Spatial demonstrates that a suite can carry both. **The finding is the gap between the declared region and the shipped states, not the ordering of the four suites.** The ordering is what the metric computes; on its own it would have been unsurprising.

What we do not claim. That a low radius *causes* a policy to memorise; that any published policy exploits it; or that admissibility on the identity-bearing object is sufficient for a constant to solve a task. §6 shows the last of those failing on our own stack, decisively.

3 When a frozen placement is exploitable

§2 establishes that one suite ships a constant where its task files ask for a distribution. This section asks when that is *fatal* — when the frozen placement is close enough, relative to what the robot can resolve, that a table indexed on the task alone reproduces whatever a perceiving policy would have supplied.

3.1 Definitions

Fix a task t that ships a finite set S_t of initial states. A task requires the policy to localise one or more objects; write O_t for that set, which the benchmark declares (LIBERO ships it as `obj_of_interest`), and $x_t^o(s) \in \mathbb{R}^2$ for object o's table position in state s. All norms are Euclidean unless subscripted; §B reports the ℓ_∞ variant, which changes one suite's verdict and not the comparison.

Definition 1 (Placement radius)**.** *The placement radius of object o in task t is the radius of the smallest disc containing every shipped position,*

$$R_t(o) \;=\; \min_{c\in\mathbb{R}^2}\, \max_{s\in S_t} \|x_t^o(s) - c\|.$$

R is the quantity the next proposition needs, and it is the quantity because a single constant serves o *if and only if* $R_t(o) \le \delta$ — the minimising c *is* that constant.

An earlier version of this paper used the *diameter* $D_t(o) = \max_{s,s'} \|x_t^o(s) - x_t^o(s')\|$. That criterion is sound where it fires ($D \le \delta \Rightarrow R \le \delta$, since $R \le D$) but it is not tight: $R \ge D/2$ always, and Jung's theorem gives $R \le D/\sqrt{3}$ in the plane. So $D > \delta$ licenses *nothing*, and we had used it to claim that no task outside LIBERO-Object was admissible.

Recomputed on R at that version's $\delta = 2.5$ cm, 27 of 38 tasks are admissible, not 10. **The separation we reported was in part an artifact of a loose bound.** What survives it is stated below, on the correct quantity and at a tolerance we measure rather than assert.

Definition 2 (Lookup-admissibility)**.** *Object o of task t is* lookup-admissible at δ *if $R_t(o) \le \delta$. A suite is* lookup-admissible *at δ if every object of every task is.*

Which objects, and why it matters. Every LIBERO task requires at least two: the thing to be moved and the place to put it. They are reported separately because the split is the finding, not a technicality. Call o *identity-bearing* if it is what individuates the task from its siblings — the grocery in "pick up the *alphabet soup* and place it in the basket" — and *shared* otherwise. LIBERO-Object's ten tasks differ only in the identity-bearing object; all ten share one basket.

An earlier version measured the identity-bearing object alone and called the suite admissible. That was measuring half of each task. Measured over both, *no* LIBERO-Object task is admissible, because the shared basket has $R \in [1.65, 1.98]$ cm and moves every episode.

Assumption A1 (δ-robustness). Supplying the controller a constant c with $\|x_t^o(s) - c\| \le \delta$ leaves the episode outcome unchanged, for every s. A1 is an empirical property of an embodiment, not a definition. §6 measures it directly and finds a radius of about 1.4 cm on this controller — and finds that the radius is not the same number on every task, which is a real limitation of what follows.

Proposition 1 (Lookup sufficiency)**.** *If $R_t(o) \le \delta$ for every object of every task in a suite and A1 holds at δ, then a policy that reads only the task index, emits the corresponding constants, and is otherwise driven by a controller whose behaviour depends on the supplied goal and not on object identity, achieves on that suite whatever that controller achieves given the true positions.*

Proof. Take c_t^o to be the centre of the minimum enclosing disc. By Definition 1, $\|x_t^o(s) - c_t^o\| \le R_t(o) \le \delta$ for every s, so $t \mapsto (c_t^o)_{o \in O_t}$ is a lookup table meeting Definition 2. A1 applies at each state, giving the same outcome as the true positions; averaging over S_t gives the claim. $\square$

	identity-bearing R (cm)		shared R	admissible
suite	mean	range	range	at 1.4 cm
Object	**0.30**	0.00–**1.17**	1.67–1.98	**10/10**
Spatial	2.15	1.49–4.49	1.66–2.01	0/10
Goal	1.48	0.00–2.48	0.00–1.70	2/10†
Long	3.10	2.85–3.28	0.00–3.17	0/10

Table 4: Placement radius of every LIBERO task, computed from the shipped initial states alone — no policy, no simulator, no training. The last column counts tasks whose identity-bearing object a single constant serves at the tolerance we measure for this controller (§6). **LIBERO-Object pins all ten of the objects its tasks are individuated by; elsewhere two of thirty**, and †both of those are the fixture tasks — a drawer and a stove — which have $R = 0$ because they are bolted down, not because a randomisable placement was frozen. On the shared container the ordering **reverses**: LIBERO-Object pins it in 0 of 10 tasks and LIBERO-Goal in 6 of 8, whose shared objects are fixtures. So LIBERO-Object is not the suite that pins the most — it is the suite that pins the object its tasks are *named* by, which is the distinction this paper is about and the reason the two columns are reported separately. Artifact: `results/admissibility.json`.

The mathematics is trivial and we do not pretend otherwise. All of the content sits in A1, which is why §6 is about measuring A1 rather than about the proposition.

Corollary 1 (No score can separate them)**.** *On a suite satisfying the hypotheses, no function of episode outcomes distinguishes the lookup policy from any policy that supplies the controller a position within δ of the truth on every shipped state.*

Corollary 1 is a statement about a *comparison class*, and the class is narrow: it contains only policies that are accurate everywhere. No policy in this paper is in it, ours least of all. So the corollary bounds what a score *could* certify; it does not license reading any particular pair of measured cells as confirming it. We flag this because an earlier version of this paper made exactly that mistake in both directions.

3.2 Measurement

Extracting a position from a shipped state is not mechanical — a naive fixed-column layout is wrong for every suite containing an articulated fixture — so we compile each task's model and read `jnt_qposadr`; Appendix A gives the procedure and the cross-check.

Two LIBERO-Goal tasks name a fixture with no free joint (a cabinet drawer, a stove). An earlier version excluded them as having "no placement to measure". That was the wrong call in the direction that flattered us: a fixture that never moves has $R = 0$ and is the *most* lookup-admissible object there is. They are counted below and identified.

Table 4 is the measurement, and it says one thing precisely.

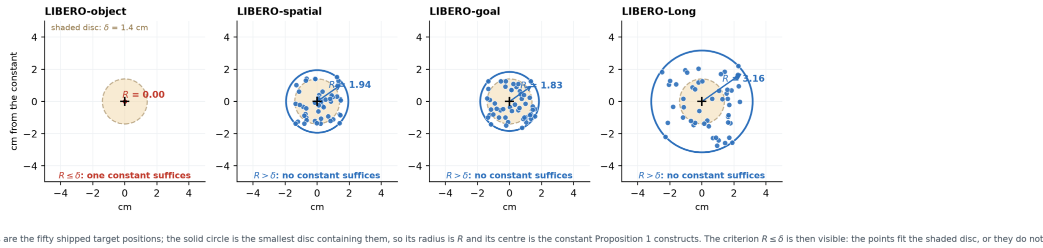

Figure 4: **Definition 1 and Proposition 1, drawn.** One task per suite, each at its suite's median radius. Dots are the fifty shipped target positions; the solid circle is the smallest disc containing them, so its radius *is* R and its centre *is* the constant the proof of Proposition 1 constructs. The shaded disc is the tolerance δ. The criterion $R \le \delta$ is then a picture: the shipped points fit inside the shaded disc, or they do not. Generated by `paper/render_fig11_radius.py`.

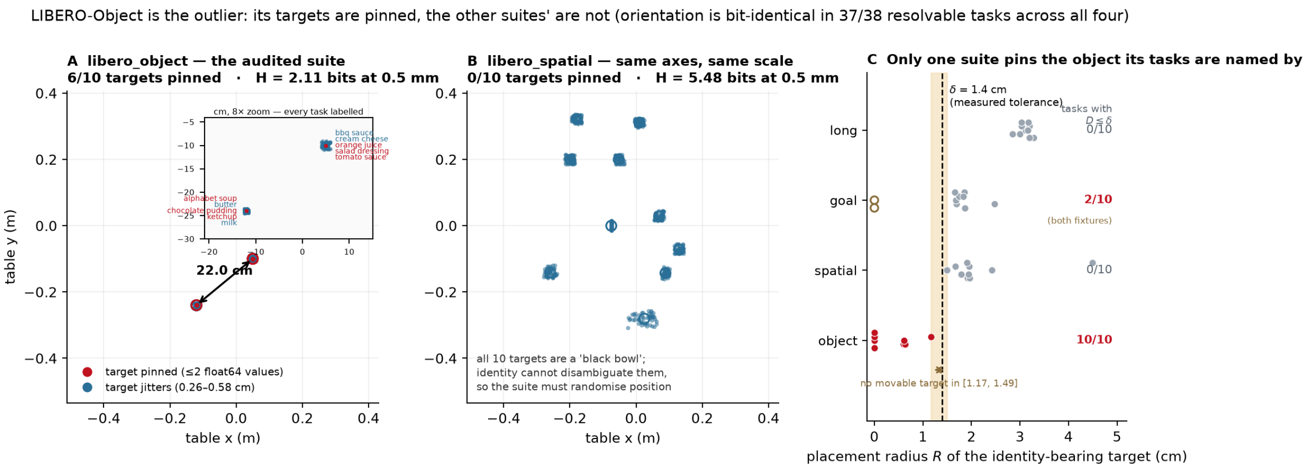

Figure 5: Placement forensics. **A** LIBERO-Object's identity-bearing targets, 10 tasks × 50 shipped states; six collapse to a point and the ten task means occupy two clusters 22.1 cm apart. **B** LIBERO-Spatial on identical axes: every target scatters, and it must, because all ten of its targets are a "black bowl" and identity cannot disambiguate them. **C** placement radius per task against the tolerance measured for this controller. Generated by `paper/render_fig1_placement_forensics.py`; SHA-256 digests of every init-state array are in `results/suite_forensics_columns.json`.

> **On the object that individuates its tasks, LIBERO-Object is lookup-admissible on all ten. Among the other thirty tasks, two are — and both are fixtures that never move at all.**

The threshold is not doing the work. LIBERO-Object's largest identity radius is 1.171 cm; the smallest among the other suites' twenty-eight *movable* targets is 1.492 cm. Every δ in that gap returns the same verdict, and the radius we measure for this controller (≈ 1.41 cm, §6) falls inside it. This is a coincidence, and we treat it as one: the window is 0.32 cm wide, and at a 2 cm tolerance most of LIBERO-Spatial and LIBERO-Goal would be admissible as well. Appendix C gives the whole curve. **The verdict is conditional on a measurement we make from a single n=10 cell**, and a reader should weight it accordingly.

How pinned is pinned. In 6 of Object's 10 tasks the target's start position takes exactly *two* `float64` values, one unit in the last place apart on a single axis — constant to the precision the format can express. The other four have $R \in [0.60, 1.17]$ cm, consistent with the sub-centimetre jitter in Figure 5A.

Across tasks the structure is tighter still: the ten targets occupy two table positions 22.1 cm apart, five tasks each (Figure 5A), so a two-constant table covers the identity-bearing half of the whole suite.

4 A glass-box stack

The artifact is an instrument, not a contribution. It is built to be traceable end to end, and its size (30.2 M deployed) is what makes an end-to-end trace feasible rather than a claim about efficiency.

Text is parsed into source/target phrases; the detector is prompted with them and returns a source box. A grasp head maps (uv, conf, proprio, box_emb, frame_emb) to a world grasp point — note the absence of any text argument, which is Figure 9's point and §9's architectural evidence. A place head maps the command embedding to a place (x, y), latched once per episode inside `reset()`

component	params	trained	deployed
YOLO-World-S detector	13.0 M	never	frozen
world model (TRM)	9.97 M	stage A	frozen, inert†
trunk heads	7.0 M	stage A	frozen
goal heads + gates	0.24 M	stage B	frozen
deployed	**30.2 M**	17.2 M trained	all frozen

Table 5: Parameter ledger. There is *no* language model: the open-vocabulary detector's own text tower supplies every text embedding, so one frozen network is the only vision encoder *and* the only text encoder. †Inert in the one cell we have ablated: a persistence baseline reproduces the held-out cell exactly. We state it for that cell only.

and never re-read. A non-learned state machine converts goals into servo commands within a $\pm 6\,\mathrm{cm}$ probe envelope.

Three words, used throughout.

- **cell** — one evaluation: a fixed (head, flags, seed, n) run to completion, always quoted with its n and its interval.
- **band** — the set of shipped initial states a cell replays. The harness fixes it; it is not sampled.
- **burned** — a band we have looked at and let influence a choice. From that moment it no longer measures generalisation, and no amount of re-running un-burns it.

Protocol. LIBERO at commit `8f1084e3`, MuJoCo 2.3.7, robosuite 1.4.1, torch 2.8.0, ultralytics 8.4.115, numpy 2.2.6, Python 3.10, `MUJOCO_GL=osmesa` — §9 explains why that last entry is not boilerplate.

Which states compose a cell is derivable from its command line, not logged. The harness sets `trial_seed` = `seed` · 1,000,003 + `trial` and indexes state `trial_seed` mod 50; since 1,000,003 ≡ 3 (mod 50), trial i at seed s replays state $(3s + i) \bmod 50$:

cell	seed, n	states
dev	0, 10	0–9
held-out	20, 10	10–19
held-out n=50	20, 50	**0–49**
untouched	40, 30	**20–49**

Table 6: Which of the 50 shipped initial states composes each evaluation cell, derived from the harness's seeding rule rather than logged. Because `trial_seed` = `seed` · 1,000,003 + `trial` and 1,000,003 ≡ 3 (mod 50), trial i at seed s replays state $(3s + i) \bmod 50$. The bolded rows are the ones that matter: an n=50 cell is the entire state set and therefore *contains* the dev and held-out bands rather than being disjoint from them.

The answer is not flattering: **the n=50 cells are not disjoint from the dev and held-out bands — they contain them.** Fifty trials over fifty shipped states is the whole set. The untouched band exists to supply one clean cell, and §8 reports it.

5 A behavioural test, and why it cannot decide

§3 says a suite with $R \leq \delta$ admits a lookup solution. Whether one actually *works* is a separate, empirical question, and this section reports our attempt to answer it.

The outcome comes first here; the previous version of this paper buried it. **The experiment does not decide the question, and we can say precisely why: the instrument tops out at 16 successes in 100 trials, eight of ten tasks score zero for every arm we ran, and the two that do score are separated by the object rather than by the goal.** What follows is the design, the result, and the diagnosis; §6.1 names the experiment that would decide it.

The vehicle is our own stack (§4): a learned goal head that emits a world grasp point, driving a non-learned pick-and-place controller. That factorisation is what makes the experiment clean — we replace the *goal supply* and change nothing else. Figure 6B lists the five arms and what each may read. Two need a word. **random** draws from the observed extent of the scene's objects, re-drawn per episode — the controller's floor. **reset-oracle** reads the true position *once* at reset and holds it; on a task with $R = 0$ that is numerically a per-task constant, but it still reads the simulator, so it is **not** Proposition 1's policy.

The floor is zero. With an uninformed goal the machinery scores 0/10 [0.00, 0.28]: eight discordant pairs, all favouring the learned head, exact McNemar $p = 0.0078$. **The controller cannot do this task without a goal.** That is the control that makes the rest of the section mean anything.

The strictly-blind policy. The **blind** arm takes the task index, opens the shipped initial-state files, and emits two constants — the mean shipped position of the identity-bearing object and of the container. On task 0 those are $(-0.1200, -0.2400, 0.0150)$ and $(0.0023, 0.2605)$.

On this task the grasp constant is not an approximation of the target position; it *is* the target position. All fifty shipped states place the alphabet soup at a single (x, y) — the fifty rows reduce to one — so whatever the blind arm fails at, it cannot be error in where it thinks the object is.

It scores 6/10 [0.31, 0.83], against the released head's 8/10. We report no inference from that pair. At n=10 an exact paired test with two discordant pairs returns $p = 0.50$, which is the largest value the test can return with any discordance at all; a power calculation gives 0.058 against the observed effect. **The cell resolves nothing, and an earlier version of this paper reported it as though it resolved indistinguishability.** It does not,

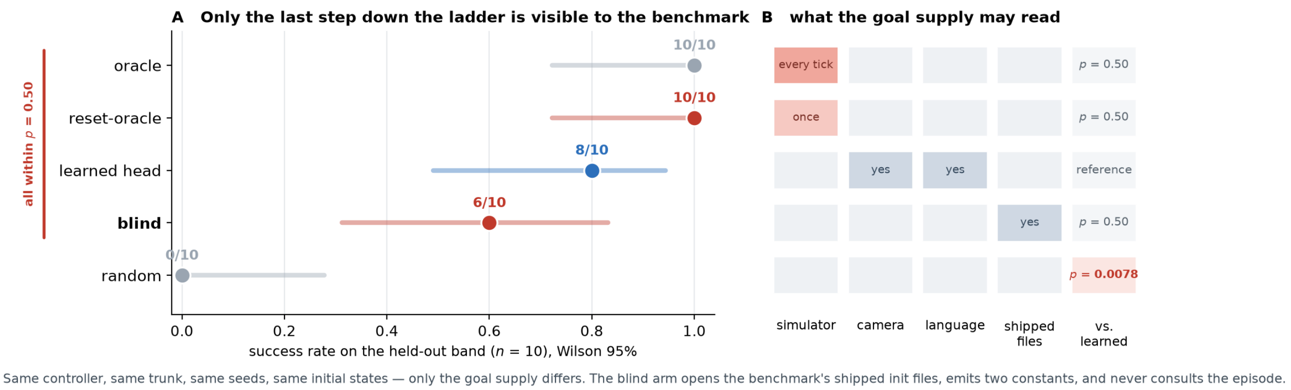

Figure 6: **The goal-supply ladder**, on task 0. Five arms, one controller, one trunk, one protocol, one set of initial states; the only difference is where the goal comes from. **B** is the experiment: descending the ladder removes information. **A** is the result — and the honest reading of it is that at n=10 this cell resolves almost nothing. The bracket marks arms the cell fails to separate, which is a statement about its power, not a finding of equivalence. Artifacts: `results/pod_cells.json`, `results/blind_cells.json`.

and the corollary does not rescue it: Corollary 1 quantifies over policies accurate everywhere, and an 8/100 policy is not one.

Three honest caveats on the arm itself, none of which we found and all of which survived into a previous draft. The constant is the *mean* over all fifty shipped states, which contains the ten this cell replays — it is fitted, very weakly, on its own test set. The arm reads the object's identity from the live scene's body registry rather than from the task index, so "reads only the task index" overstates it. And it requires a live simulator to start at all.

The *control* claim survives all three — with an anchor set, the goal machine is driven by proprioception and the constant, and the grasp head is never called — but the *blindness* claim as we first wrote it did not.

6 The diagnosis

Every cell above is one task. Run on all ten at seed 20 — so every cell replays states 10–19, the held-out band of Table 6; that band is burned for task 0 by the selection loop of §7 and untouched for tasks 1–9, which no round of selection ever reached — the blind constant scores 16/100 and the released head 8/100. An earlier version of this paper led with that pair, paired the 100 trials, ran an exact McNemar, obtained $p = 0.039$, and concluded that a task-indexed constant significantly beats a trained vision-language policy.

That inference is wrong, and the error is instructive.

The unit of analysis. All twelve discordant pairs live in two of ten tasks, and ten of them in one (Table 7). Within that task the ten "trials" are near-identical replays: blind terminates at steps 210–214 (sd 1.08), the head runs to the 300 cap on all ten. They are one deterministic contrast counted ten times.

At the task level — the unit the design actually randomises over — blind wins one task, loses one, and ties eight: exact sign test $p = 1.0$, exact sign-flip permutation $p = 1.0$, cluster bootstrap on the difference $[-0.06, +0.30]$. The design effect is 8.4, so the effective n is about 12, not 100.

unit	test	result	
trial (n=100)	exact McNemar	10–2, $p = 0.039$	*invalid*
task (n=10)	exact sign	1–1, 8 ties, $p = 1.0$	valid
task (n=10)	permutation	$p = 1.0$	valid
task (n=10)	cluster bootstrap	$[-0.06, +0.30]$	valid

Table 7: The same comparison at two units of analysis. Trials within a task share an object, a scene, a goal supply and a near-deterministic trajectory, so the trial-level test is anti-conservative. **The task-level tests do not show equivalence — they show this design cannot resolve the question in either direction.** With eight ties the sign test has two informative units and its smallest attainable p is 0.5; the permutation test has four arrangements and the same floor. Reading $p = 1.0$ as "no difference" would repeat, in the other direction, the error this section withdraws. The design effect (≈ 8, bootstrap $[5.4, 10.0]$, estimated from one non-degenerate cluster) is reported as a description of the clustering, not as a correction to any p. Artifact: `results/suite_inference.json`.

The constant did not produce the successes. This is the sharper finding, and it comes from the same run. An earlier version compared the ten constants *in the table plane* and said they were identical to within a millimetre. The arm is handed a 3-vector, and in three dimensions the two position groups differ by 40.0 and 35.0 mm on z. That sentence was true in 2D and false as written.

What survives is one comparison, and it needs no statistics. **Tasks 2, 5 and 9 are handed a constant 0.469 mm from task 3's in full three dimensions.**

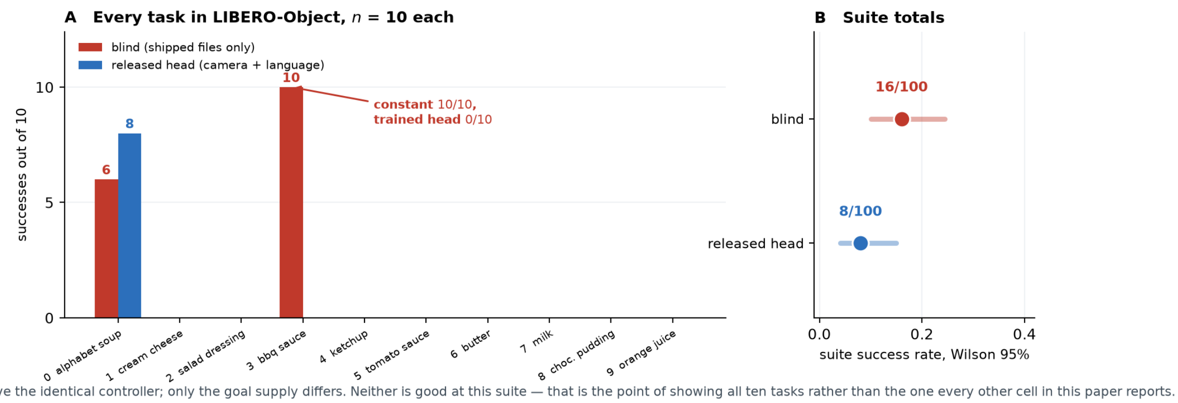

Figure 7: **All ten LIBERO-Object tasks**, n=10 each, every trial recorded; both policies drive the identical controller. Eight of ten tasks are 0–0. The suite totals in **B** are shown with i.i.d. Wilson intervals, which are *too narrow* here by a design effect of 8.4 (§6); they are drawn as reported rather than as believed. Artifacts: `results/suite_cells.json`, `results/suite_inference.json`.

They score 0/10**. Task** 3 **scores** 10/10**.** Half a millimetre of goal, against a tolerance of centimetres, cannot separate a total failure from a clean sweep.

task		distance from task 3's constant (3D)	blind
3	`bbq_sauce`	—	**10/10**
2	`salad_dressing`	0.469 mm	0/10
5	`tomato_sauce`	0.469 mm	0/10
9	`orange_juice`	0.469 mm	0/10

Table 8: **The goal supply does not explain the outcome.** Four tasks receive the same constant to within half a millimetre in all three axes and score 0, 0, 0 and 10 out of ten. This is a deterministic contrast, not a statistical one. We previously also offered a correlation between placement radius and success ($r = +0.52$) as evidence here; **we withdraw it**. Its exact permutation p is 0.13 and deleting task 3 flips it to -0.25 — the sign is one task, the error this section reports. Artifact: `results/constant_attribution.json`.

What varies across those tasks is the object, not the number. So the blind arm's two non-zero cells are evidence that *this controller can grasp two of ten groceries*, and not evidence about lookup. **We withdraw the suite-level claim.**

Where A1 does break, we can see it. One place in this data does behave the way the proposition expects. On task 0 the blind arm reaches the object to within 2 mm on all ten trials — the grasp constant is exact there — and the four failures are the four trials whose *basket* sits furthest from the constant, occupying ranks 6, 8, 9 and 10 of ten (exact Mann-Whitney $p = 0.019$, Figure 8). Task 3 tolerates 1.91 cm and never fails. That gives the ≈ 1.4 cm radius §3 uses, and shows it is task-dependent.

Two caveats we owe a reader, because this is the number §3 leans on. The hypothesis was formed *after* a preregistered one failed, on the same ten points. And the covariate is confounded with trial order: the outcome is exactly $\mathbf{1}\{\text{trial} \le 5\}$, trial i replays state $10 + i$, and displacement correlates with state index at $\rho = 0.65$, so trial index alone predicts better ($p = 0.0095$). At n=10 the design cannot separate them.

6.1 What would settle it

Displace the constant by r and sweep r: if the score is flat in r, the arm never used the number. We implemented it (`--goal-anchor-jitter-cm`) and the compute window closed before it ran. Until it does, the strongest statement the suite supports is the negative one above.

What survives. Two things, First, on task 3 a constant scores 10/10 where the trained head scores 0/10 — a single task-level event, so we quote no p for it, but one worth stating: the head is not merely worse there, it never succeeds.

Second, and independent of any of this, **eight successes in a hundred trials, every one on the single object the head was trained for**. Our artifact is a one-object solution that a suite-level score would have reported as a policy. The number a reader should carry away is not 0.700; it is 0.700 on task 0, with 0.000 next to it nine times.

And it bounds the audit in advance. Whatever §7–§8 bought, it was not a general policy. We put this section before the audit so that no reader reaches the repairs holding a larger idea of the artifact than the artifact supports.

7 The audit: four layers

"The policy memorised the location" is not one hypothesis but four, at four stages — causal confusion [1] spread across a pipeline rather than confined to a network — two of which are not in the model at all — and those

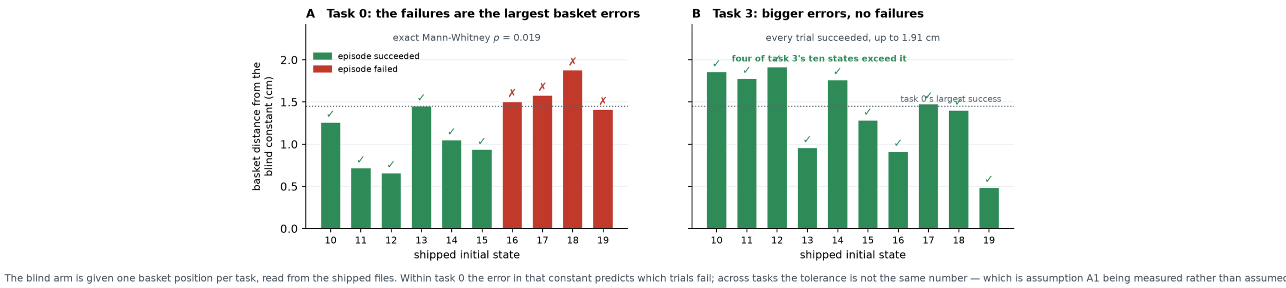

Figure 8: **The one place the constant demonstrably matters.** **A** On task 0, the trials that fail are those whose basket sits furthest from the single constant the blind arm was handed. **B** On task 3 the same quantity runs higher and every trial succeeds, so the tolerance is not one number across tasks. This is the measurement §3's $\delta \approx 1.4\,\mathrm{cm}$ comes from; see the confound caveats in the text. Artifact: `results/blind_failure_attribution.json`.

two are the ones a black-box perturbation study cannot reach.

Three of the four are described here; the fourth, being the one the instruments found rather than confirmed, gets its own subsection.

Layer 1: the expert's calibration constant. The scripted demonstrator carried a hand-eye offset fitted once, on a scene whose target never moved. That constant therefore *encoded* the pinned placement, and every demonstration inherited it — so the corpus taught the pose before any network saw a pixel. Detected by reading the expert, not the policy. Repaired by re-baking ten episodes with the source object displaced, which breaks the constant's validity and forces the demonstrator to look; the repaired corpus lifts the dev cell from 0/10 to 7/10 on the same episodes.

Layer 2: the selection loop. Our own procedure, not the artifact's: at each round we chose a checkpoint using cells that later rounds reused, so selection pressure accumulated on one band of initial states. This is invisible to any measurement of the model and is the reason §8 runs an untouched band at all. It is *bounded* rather than repaired — one cannot un-look at data — and the bound is that thirty never-inspected states score within noise of the burned ones.

Layer 3: the grasp head's proprioceptive shortcut. Substitution attribution showed the predicted grasp point tracking the end-effector and barely moving under *uv* substitution: the head had learned to predict where the object is from where the arm is, which on a pinned suite is a sufficient statistic. The same teleported episodes repair it, and the released head's profile inverts — it now moves further on its visual channels (3.43/6.57 cm) than on proprioception (1.93 cm).

7.1 Layer 4: where the language goes

Swapping the instruction while holding the scene and the success predicate fixed collapses the released head from 8/10 to **0/20** $[0.00, 0.16]$; on the ten trials that pair with the reference cell, eight discordant pairs all one way, exact McNemar $p = 0.0078$.

Telemetry places the failure *downstream of approach*: the policy still reaches the true object as closely as baseline while the grip-close rate falls from ∼0.67 to ∼0.26. Driving the detection prompts and the place-head embedding from *different* instructions isolates the collapse to one channel, with no interaction.

Combined with the architecture: **object selection, approach and grasping run with no language input at all, and the entire language channel is a single cached** (x, y). Ask this policy for the butter and it finds, approaches and grasps the alphabet soup at baseline rate; it fails only when that one coordinate is wrong.

8 Repairs, and one we withdraw

Three of the four layers admit a repair, and we report what each bought. One claim from the previous version does not survive being run at power, and we retract it here rather than leave it standing.

The untouched band. Both n=50 cells contain the burned states, so we ran the released head on states 20–49, which no selection look ever reached, with the prediction recorded before the run ($[0.50, 0.75]$, falsifiers named in both directions). Result: **20/30** $=$ **0.667** $[0.49, 0.81]$, against the published 0.700 (35/50) a difference of -0.033, Newcombe $[-0.24, +0.16]$. That interval assumes independent samples and the samples are *not* independent: the n=50 cell spans states 0–49 and so contains all thirty untouched states.

The interval is therefore conservative in the wrong direction, and we use it only to bound the effect as small. The clean comparison — untouched-30 against the twenty burned states — is the one we would run next.

layer	where	evidence	repair	verification / status
1. expert calibration constant	scripted expert (non-model)	hand-eye constant encodes the pinned placement	placement teleportation	0/10 → 7/10 dev, paired. **Repaired**
2. selection loop	our own procedure (non-model)	checkpoints chosen on cells later rounds reused	process fix; bands frozen	untouched band 20/30 vs 0.700: no detected inflation. **Bounded, not removed**
3. grasp-head proprio.	`GraspPointHead`	prediction tracks the eef, flat under uv sweeps	same teleported episodes	attribution flips to visual. **Repaired**
4. place-head cmd→location	`PlaceHead`	correct basket for the trained command, 14.5/13.0 cm off for the other two; swap 8/10 → **0/20**, p=0.0078	command coverage	place *point* 14.15 → 0.78 cm **repaired**; swap *behaviour* 0.767 → 0.433, p=0.031 — **halved, not fixed**
(separate) appearance drift	deployment viewpoints	NN-cosine gap on the policy's own trajectory	DAgger on own viewpoints	3/50 → 22/50 vs 3/50 control, $p < 10^{-4}$. **Repaired**

Table 9: Master audit table. Layers 1–4 are the placement-memorisation layers; the last row is a drift case study, *not* one of the four; it appears here only because its repair is easily mistaken for one of theirs. Two rows are deliberately not marked repaired.

We claim exactly this much: not that the burn is zero, but that it is small enough to hide inside ±0.20, which is what n=30 buys.

A repair we withdraw. Command coverage was reported as repairing the instruction-swap behaviour on the strength of 3/10 against a 4/10 baseline at $p = 1.0$. That is an equivalence claim from an underpowered cell, on a band where the head was already mostly failing — and a swap cannot visibly break a cell that is already broken. Re-run on a band where the coverage head scores well, and taken to n=30:

head	baseline	swapped	paired McNemar
released head	8/10	**0/20**	8–0, $p = 0.0078$
coverage	23/30	**13/30**	12–2, $p = 0.031$

Table 10: Substituting a different task's instruction, on a band where each head scores well and at a power the original cell did not have. Both heads lose accuracy, so command coverage does not eliminate the instruction dependence it was built to remove; it halves it. Paired McNemar over matched trial indices (same seed, same initial states, same renderer — only the instruction differs), exact two-sided.

At n=10 this contrast was $p = 0.125$ and we would have called it unresolved; at n=30 it is $p = 0.031$. **The coverage head collapses too.** A swap cannot visibly break a cell that is already mostly broken, which is why the original 4/10 baseline hid the effect.

What replaces the withdrawn claim is more useful than it was. Coverage does not eliminate the failure, it roughly *halves* it: the released head loses its entire cell (0.800 → 0.000, every discordant pair one way), the coverage head about a third (0.767 → 0.433). And the repair to the place *point* is not in doubt — 14.15 → 0.78 cm (Figure 10), measured directly rather than inferred from behaviour.

So: **command coverage fixes where the head aims and does not fully fix what it does.** A residual command→location dependence survives training on all three commands — a sharper statement of layer 4 than we had, and one no cell we ran before could have detected.

8.1 Randomisation is a curve, not a point

Our ±4 cm radius was chosen to sit inside the controller's ±6 cm probe envelope — a perturbation calibrated to the system under test. Sweeping it on the same draws separates two things that single number confounded:

displacement	released head	Wilson 95%
none‡	4/10	$[0.17, 0.69]$
±2 cm	7/10	$[0.40, 0.89]$
±4 cm‡	4/10	$[0.17, 0.69]$
±6 cm (envelope edge)	2/10	$[0.06, 0.51]$
±8 cm (outside)	3/10	$[0.11, 0.60]$

Table 11: Sweeping the placement-randomisation radius on the same draws. Randomisation robustness is a curve, and at n=10 this curve is not even monotone — the undisplaced cell sits *below* ±2 cm, which cannot be a real benefit of displacement. Reporting any single radius as "passes randomisation" would quote a number this sample does not resolve, and we do not use this table to set any threshold elsewhere in the paper. ‡Per-trial outcomes for the ±2, ±6 and ±8 rows are in `results/pod_cells.json`; the two marked rows were measured in an earlier session and their trial records are not in the repository.

Read conservatively, because n=10 supports no more: clearly worse at ±6–±8 than at ±2, but every interval

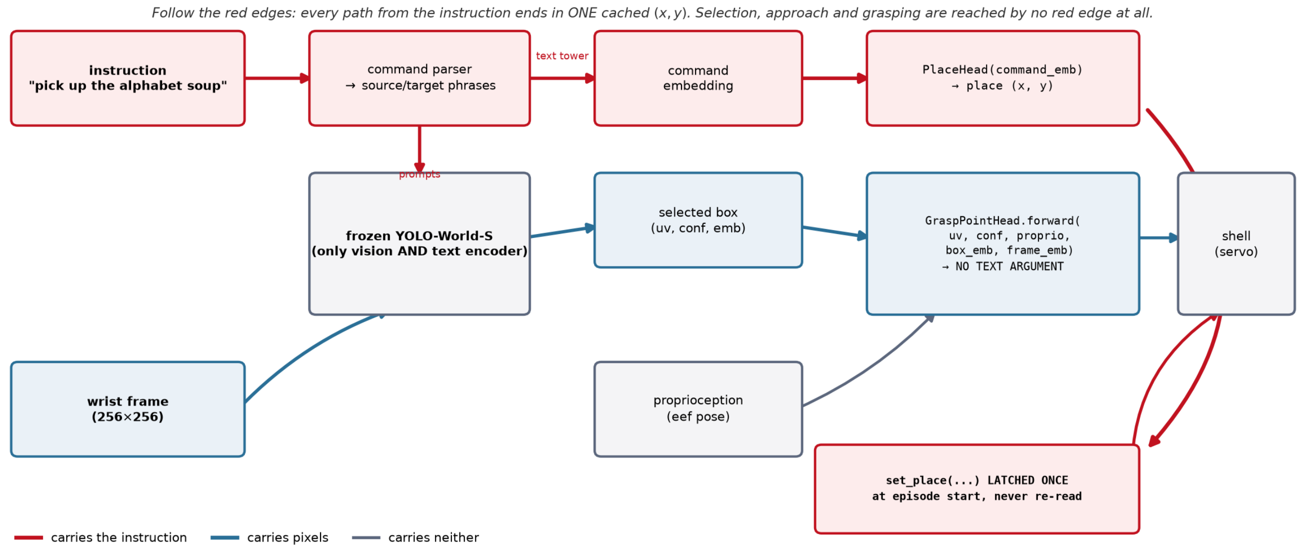

Figure 9: Every edge coloured by what it carries. The architectural half of the argument is a code fact: `GraspPointHead.forward` takes no text argument, so no red edge reaches object selection, approach or grasping, and every red edge terminates in one (x, y) latched once per episode and never re-read.

is ∼0.5 wide and the undisplaced cell sits *below* ±2 cm, which cannot be a real benefit of displacement. That non-monotonicity is the honest measure of what this n resolves. **No single radius should be quoted as "passes randomisation"**, and a benchmark asking for one is asking for a number that does not exist.

head	uv	proprio	box_emb	frame_emb
v2	0.75	2.85	6.04	6.91
v2.1	0.71	2.13	6.26	7.58
v5 (released)	0.97	1.93	3.43	6.57

Table 12: Substitution-probe attribution, in cm of movement of the predicted grasp *offset*, per input channel. The counterexample is the first two rows: v2 and v2.1 agree to within 0.73 cm on every channel and score 0.000 and 0.700 when deployed — these are heads v2 and v2.1, an earlier pair, and their 0.700 is not the released head's 35/50. A probe evaluated on teacher trajectories cannot, on its own, certify off-manifold behaviour.

9 Certification

If a benchmark score cannot certify grounding, what can? This section reports three results about the certification problem itself, two of them negative.

9.1 Probes alone cannot certify

A substitution probe asks which input channels a head reads: swap one channel between two recorded ticks and measure how far the predicted grasp offset moves. It is annotation-free, cheap, and on-manifold — and that last property is fatal on its own.

One detail in Table 12 decides whether the probe means anything: it profiles the grasp *offset*, not the absolute point. The head emits $\mathrm{eef}_{xy} + \Delta$, so substituting proprioception moves the absolute prediction by the entire arm displacement for *any* head, and profiling that quantity reports ∼24 cm of "proprioception attribution" even for a repaired one.

We made exactly this error when regenerating the measurement. It is the difference between a probe that answers "does this channel change where the head thinks the object is" and one that measures the parameterisation.

v2 and v2.1 differ by at most 0.73 cm on any channel and score 0.000 and 0.700 deployed. One caveat we state rather than let a reader discover: the substitution is between a *single* pair of recorded ticks — the two most separated in *uv* out of 329 — with no resampling over pairs and therefore no interval on the 0.73 cm. It is an existence proof that two heads can be probe-identical and behaviourally opposite, not an estimate of how often that happens.

A probe evaluated on teacher trajectories is an on-manifold instrument and cannot, alone, certify off-manifold behaviour. Probe evidence must be paired with behavioural randomisation.

9.2 An instrument only ever shown firing is not yet an instrument

Our cross-instruction detection-agreement probe asks whether two objects' prompt chains select the *same box from the same frame*; a high same-box rate says the grounding stage carries no object identity. Every pub-

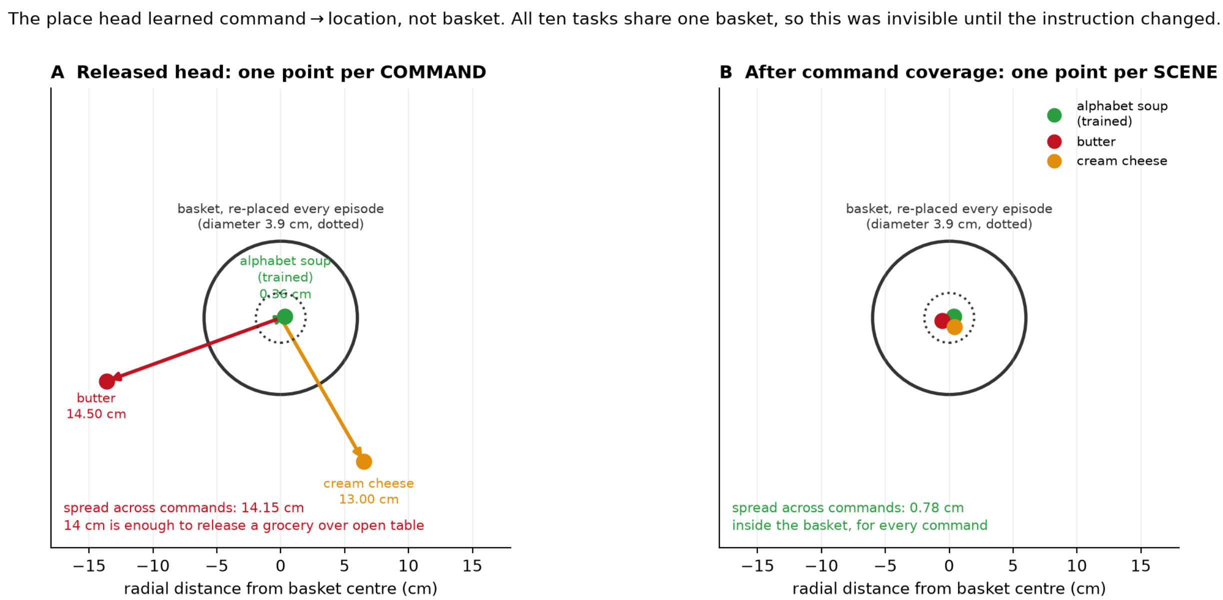

Figure 10: Layer 4 drawn rather than argued. The released head asked for three objects emits three place points, 14.5 and 13.0 cm from the basket. The basket is *not* fixed — it is re-placed every episode, with a placement radius of 1.83 cm averaged over the suite's ten tasks — so a memorised place point is wrong by 14 cm against a target that moves by under 2, and 14 cm is enough to release a grocery over open table. **Only radial distance is measured**: the angles are chosen for legibility and carry no information, which is why no coordinate grid is drawn.

lished number from it was a firing. Running it against every other object in the scene splits cleanly:

counterpart	n	same-box	median dist.
cream cheese, butter, pudding	6	**0.00**	0.817–0.818
tomato sauce	4	1.00	0.00069
bbq sauce, salad dressing	5	0.80	0.0043
ketchup	5	0.60	0.0048

Table 13: The positive control our detection-agreement instrument had never been given. "Same-box" is the fraction of frames on which two objects' prompt chains select the identical box; a high rate means the grounding stage carries no object identity. Run against *every* counterpart rather than only the ones that fire, the instrument separates cleanly — and the split is interpretable: the chains that collide are the cans and bottles, the ones that separate are the boxes. Deployed binding is blind *within* a shape class, which is the regime LIBERO-Object's groceries occupy. Small n (only 4–6 of 60 frames have both chains detecting) makes these demonstrations of discrimination, not calibrated rates.

The instrument registers difference when difference exists. The distances are bimodal — collisions at $\sim$0.004, separations at $\sim$0.82, nothing between — and a sweep over $\tau \in \{0.005, 0.01, 0.02, 0.05\}$ leaves every verdict unchanged. The split is interpretable and sharpens the claim: the chains that collide are the cans and bottles, the ones that separate are the boxes. **Deployed binding is blind *within a shape class***, which is precisely the regime LIBERO-Object's groceries occupy.

Two limits. Only 4–6 of 60 frames had both chains detecting, so these are small-n demonstrations of discrimination, not calibrated rates. And the contrast we originally designated the positive control — the basket — is not evaluable at all: the deployed source-area filter rejects basket-sized boxes by construction, so it compares zero frames. Our first version of the script read same_rate = 0 off that empty comparison and printed "instrument discriminates". A verdict now requires $n \geq 3$.

9.3 Certificates are joint in (head, stack), down to the renderer

We rebuilt the deployment stack on fresh hardware, pinning every version in §4 and verifying both load-bearing checkpoints by digest. The reference cell reproduces: 8/10 $[0.49, 0.94]$ against a recorded 7/10, with 0.700 inside the interval.

Reproducing the *rate* is not reproducing the *experiment*. Holding seed, trial, weights and every package version fixed and changing only the OpenGL backend MuJoCo renders through:

The aggregate is identical and 40% of the episodes are not — but we can no longer attribute that to the backend *change*.

MuJoCo's software and EGL renderers are documented as nondeterministic *within* a backend, so repeated renders of one state need not agree, and a 4/10 disagreement is consistent with that alone.

The control this needs is osmesa against osmesa at the full horizon. The one we have runs 40 steps and stops before the gripper closes — the contact phase where any

	`osmesa`	`egl`
success rate	8/10	8/10
per-trial outcomes	**4 of 10 disagree** (2 each way)	

Table 14: Changing only the OpenGL backend MuJoCo renders through — same seed, same trials, same weights, same package versions. The aggregate is identical and 40% of the individual episodes are not. Matching a published rate is therefore weak evidence of having reproduced an experiment, and any paired statistic computed across a renderer change is invalid, because pairing assumes the trials are the same trials.

such divergence would show.

We report the disagreement and withdraw the causal reading — the same error this paper's conclusion warns against. Thread count is a remark rather than a result, for the reason just given: three `osmesa` runs at 1, 4 and 128 threads agree on all eight logged fields while the wall clock falls from 220 s to 32 s. We state the limit plainly — that comparison ran to 40 steps, a prefix in which the gripper never closes, so it does not probe the contact-rich part of an episode where the renderer difference showed up (`results/thread_determinism.json`). Software and GPU rasterisation do not produce identical pixels, the detector is not invariant to that difference, and in this stack the detector is the only encoder.

Two consequences. Matching a published rate is weak evidence of having reproduced an experiment: the marginal can be right while the sample is different. And *any paired statistic computed across a renderer change is invalid*, because pairing assumes the trials are the same trials.

We must therefore report our own compliance honestly. Every paired test in §6 is within one build, because those cells were produced in one sweep. We cannot say the same for the whole paper: the confound test of §11.1 ran on a different machine entirely — macOS, MuJoCo 3.3.0, robosuite 1.4.0, Python 3.13, recorded in its artifact — and no per-cell build ledger exists for the older cells. Given what this subsection demonstrates, that is a gap in our own protocol and not a footnote. `MUJOCO_GL` belongs in any reported stack; it was not in ours, or in any LIBERO evaluation we have read.

10 Transfer: a negative result

The instruments above are offered as portable, so we tried them on a policy we did not build: the public `openvla-7b-finetuned-libero-object` checkpoint [6], through the same harness, trial-to-state map and success predicate.

The baseline returned 0/1 on a checkpoint published near 0.9. We treated that as a harness defect and found four: image orientation, the gripper convention (their model emits $[0, 1]$ with 1=close, robosuite wants $[-1, +1]$ with -1=close — without the conversion the jaw never closes), success extraction (the environment's `done` flag also fires on horizon), and a missing signal shield. We then swept four image orientations against the 0.9 centre crop their evaluation applies.

The checkpoint still does not approach the target in any configuration we tried. **We therefore report no OpenVLA numbers at all** — not success rates and not distance statistics — because the diagnostic runs were exploratory and were never written to a versioned artifact, and an unvalidated baseline would make our instruments look powerful for the wrong reason.

All four defects biased in the same direction — toward the checkpoint looking worse — which is precisely how an external-validation result gets fabricated without anyone intending it. We cannot separate a remaining defect in our harness from a checkpoint that does not transfer to a differently-built LIBERO, and shortfalls in reproducing this checkpoint's published LIBERO-Object number are reported by others on the project's issue tracker, so we treat the gap as unexplained rather than as a finding of ours. The harness ships (`eval/openvla_eval.py`). Until someone with the matching build runs it, **the portability of these instruments is unestablished**.

11 Negative results and registries

A paper that only reports what worked cannot be checked. This section is the ledger: what we predicted before running, what failed, which instruments we retired, and the three ways our own results fooled us.

Pre-registration. Fourteen predictions are on record with their outcomes; seven failed. Two failures matter here: the coverage head's swap cell did not hold (p=0.031), and the released head was not at floor outside the controller's envelope (3/10).

The blind arm carries *no* registered prediction — it was implemented in one compute window and run in the next — and neither does the attribution analysis that overturned it (§6), which was formed after the registered hypothesis failed. Neither is reconstructed after the fact.

Retired cells. Two cells from earlier rounds are retired rather than silently dropped. An n=50, $\pm$4 cm displacement cell scoring 0.520 is superseded by the per-radius curve of Table 11, which resolves the shape of the response where a single radius could not. And a free-regression variant scoring 0/56 is withdrawn: it was measured on a configuration this paper no longer describes, and we have no per-trial record for it.

Retired instruments. Quantifying deployed role binding was attempted six times and abandoned six times, each at a calibration gate written down before the instrument was trusted. Deployed binding accuracy is reported as **unmeasured**. The six: a projected-origin ground truth (the object that succeeds 35/50 scored 0.111), its sign-corrected successor (the in-frame filter deletes the close-up target by construction), three text-tower discrimination probes (each failed to detect the working object's own phrase), and a box-spread metric (confounded by camera motion, and inverted).

11.1 A confound we had missed

Our own analysis of a blocked object was confounded: because the suite's two placement clusters align with which objects we happened to train on, "blocked" and "22 cm from the training position" were perfectly correlated in our design. Tested directly over all ten objects, position predicts the effect weakly (Spearman +0.19, p=0.60) and *teleporting each object* 22 *cm shows no directional effect* (6/10, exact sign test p=1.0, the least informative value the test returns), while detector groundability predicts it (−0.71, p=0.027). The mechanism is appearance, not position — position is a nuisance term with mean $|\Delta| = 0.0066$, which is not zero either.

This does not contradict §3, and the distinction is worth stating. Those sections answer different questions. Placement is what the suite fails to randomise and therefore what a policy *may exploit*: that is a property of the benchmark, established from its shipped files, and it is what layers 1 and 3 were caught doing. Appearance is what limits *transfer to a new object* once that shortcut is repaired: a property of the frozen detector, and why the released head works on one object and not on nine (§6).

A benchmark can be lookup-admissible and a policy can still fail for an unrelated reason. Both are true here, neither weakens the other, and what *would* have been a contradiction — position explaining both — is what the confound test ruled out.

We also report the caveats on this cell rather than only its conclusion. Both nulls are n=10, which is the size we decline to accept elsewhere (§8); the groundability predictor and the outcome are two statistics of the same detector forward pass, so their correlation is closer to a self-consistency check than to an independent prediction; and $p = 0.027$ across three tests does not survive a Bonferroni correction. The confound is *disconfirmed as a sufficient explanation*, which is what the design can support, and not more.

11.2 Four ways a result fooled us

Recorded because the failure modes generalise and *none* was caught by a conventional check — in all four the instrument was correct and the defect was elsewhere (Table 15). The fourth is this revision's headline.

what we saw	what it was	the general lesson
a perfect null: teleport gaps identical to control for all ten objects, to four decimals	a cached observation — the renderer returned the *pre*-teleport frame	implausible cleanliness is a symptom. The script now compares frames across the intervention and *raises* rather than emitting
a defect count withdrawn as unsupported	the appendix we could not find existed	"I could not find it" is a claim about a search, not about a corpus
7/7 against a published 7/10; we began writing up a failure to reproduce	a censored sample. The complete cell is **8/10**	successes end episodes early and failures run to the horizon, so any interim harvest is biased toward successes. **Verifying the instrument does not verify the sampling**
a constant outscoring our trained policy 16/100 to 8/100, $p = 0.039$	ten replicates of one task counted as ten trials — and a constant that did not cause the successes at all	we checked the numbers and not the *unit*, nor whether the variable we manipulated was the one that moved. **Both errors were found by adversarial review of data we had already published**

Table 15: Four self-inflicted errors and their repairs. The third is the one we would most expect others to hit: every check we ran had passed, and the defect was in *when* we looked. The harvester now reports each cell against its planned n and refuses to report an incomplete one.

12 Discussion

What a score certifies. On a suite whose placement radius sits below the embodiment's tolerance, a success rate certifies that the policy can execute a manipulation *given* a correct target pose — and nothing whatever about how it obtained one. That is a statement about what the score *could* distinguish, and it is exactly as strong as A1.

The narrow statement is deliberate: we already overreached once. Our own cells do not demonstrate that any policy exploited the degeneracy; §6 shows they cannot. What the radius buys is a check that runs before training, and a prediction — that a suite with small radii admits a cheap solution — which Jiang et al. [5] confirm on LIBERO with a trained probe where we did not with a constant.

Shortcuts live in pipelines, not only in models. Two of our four layers are outside the network: a scripted expert's calibration constant and our own checkpoint-selection loop. No amount of perturbing a trained model from the outside would have found either. This is an argument for glass-box artifacts in benchmark-validity work, and it is the main reason our stack is small.

Certification is a system property. Probes cannot certify alone (two heads, 0.73 cm apart, 0.000 vs 0.700). Behaviour must be paired with randomisation. And the certificate is joint in *(head, stack)* down to a component no requirements file pins: the same weights on the same seeds give the same rate and a different 40% of episodes under a different renderer.

Limitations.

- One simulator, one benchmark suite, one robot, wrist camera only.
- Every confirmatory cell is one task and one object at $n \leq 30$, including the blind arm (n=10). The *radii* of §3 are computed from files and carry no such limit; the *verdict* does, because it needs δ.
- One 10-trial cell (the released head on task 0, seed 20) appears in five reported results: the suite total, the blind comparator, the renderer table's `osmesa` arm, the instruction-swap baseline, and the reproduction check. It is one sample, not five.
- The held-out band is partially burned; the disclosure is in §4 and the bound in §8.
- The on-manifold limit of substitution probes is a limit on our own instruments, not only on others'.
- Transfer to an external policy is **unestablished** (§10).
- R is defined for suites shipping enumerable initial states, and would need restating for procedurally generated ones.

12.1 Recommendation to benchmark authors

Both quantities are cheap to compute and cheap to report. Table 16 is what we would ask of a suite.

practice	cost	why
publish R per object per task	one script, no training	tells a reader what a score can certify
set a floor on R	re-sample the init files	a suite individuated by identity must randomise placement, or identity is never load-bearing
ship a tier of radii	k extra eval configs	where a policy degrades depends on displacement relative to the controller's envelope (Table 11); one radius is a calibration, not a result

Table 16: Three practices, in increasing order of what they cost a benchmark's authors. LIBERO-Spatial already satisfies the second by necessity — all ten of its targets are the same bowl — and LIBERO-Object could satisfy it by choice.

13 Conclusion

A manipulation benchmark score is not a measurement of capability until the benchmark's admissibility has been measured. The measurement is cheap, it runs before any model is trained, and on LIBERO it says something specific: the one suite whose tasks are individuated by object identity is also the one that pins that object, on all ten tasks, to a radius smaller than the tolerance we measure for our own arm.

The behavioural claim we built on that does not stand, and the way it fell is the more useful half of this paper. We ran the lookup policy, saw it outscore our trained one, tested it at the wrong unit of analysis, and reported a significance that a task-level test does not support. The deeper error was attributional: tasks handed constants identical to within a millimetre score from 0/10 to 10/10, so the successes were never evidence about lookup at all. Both errors were found by adversarial review of a manuscript we already believed, using data we had already published.

What survives is the procedure rather than the artifact: measure the benchmark first, at the unit the design randomises over; check that the variable you manipulated is the one that moved; and report the reading that did not survive being checked, which in this revision is our own.

A Recovering which column owns which object

LIBERO ships each task's initial states as a MuJoCo flattened state $[t \mid q \mid \dot{q}]$; every shipped $\dot{q}$ is zero, so the columns that vary across states are exactly the position degrees of freedom the suite randomises.

Recovering *which* object owns which column requires care. The fixed layout $[t \mid 9 \mid N \times 7 \mid \dot{q}]$ that works for LIBERO-Object predicts a width of $19 + 13N$ and fails for every suite containing an articulated fixture — LIBERO-Spatial ships width 92 against 5 declared objects. We therefore run two passes: the fixed-layout one, and one that compiles each task's model and reads `jnt_qposadr` with the joint names, mapping every column onto a named body. **The two agree wherever both apply**, and only the second is used for suites the first cannot parse. Artifacts: `results/suite_forensics_columns.json` (digests), `results/suite_forensics_joints.json` (per-object positions).

B The norm, recomputed

Definition 1 uses the Euclidean norm; A1 is naturally stated in ℓ_∞, because LIBERO randomises placement in an axis-aligned box. A previous version of this appendix asserted, without computing it, that the switch changed one LIBERO-Spatial task and left the comparison intact. That was wrong, and a referee caught it. Computed (`results/admissibility.json`), at $\delta = 1.4\,\text{cm}$:

suite	ℓ_2	ℓ_∞
Object	10/10	10/10
Spatial	0/10	0/10
Goal	2/10	**3/10**
Long	0/10	0/10

Under ℓ_∞ the "both are fixtures" clause fails. The third LIBERO-Goal task to cross is `put_the_bowl_on_the_stove`, whose bowl moves. The ℓ_∞ separating window is $[0.97, 1.38)$ cm, which *excludes* the $\delta = 1.41\,\text{cm}$ we use. So in the norm we ourselves call natural for A1, the headline does not hold. We report the Euclidean version because it is the one our δ was measured in, and we flag that this is a choice the result is sensitive to.

C Sensitivity of the verdict to δ

This appendix is where the paper's headline is most vulnerable, so we give the curve at fine granularity rather than at the points that suit us. A previous version of this table stepped from 1.49 directly to 2.0 cm, which happens to skip the interval containing our *own* second tolerance estimate. That was not deliberate and it was not defensible.

δ (cm)	Object	elsewhere	
1.17	9/10	2/30	
1.40	**10/10**	**2/30**	← task-0 estimate
1.49	10/10	2/30	
1.60	10/10	3/30	
1.70	10/10	7/30	
1.80	10/10	9/30	
1.91	10/10	**13/30**	← task-3 estimate
2.00	10/10	17/30	
2.50	10/10	19/30	

Table 17: Tasks whose identity-bearing object one constant serves, against the tolerance. **The separation is real at 1.4 cm and gone by 1.9 cm**, and those two numbers are our own two measurements of the same assumption (§6): task 0 fails at basket displacements from 1.41 cm, task 3 tolerates 1.91 cm. We report the headline at the smaller one, which is the value that favours us. At the larger one LIBERO-Object is still 10/10 but so are thirteen tasks elsewhere, and the contrast is no longer a separation. Artifact: `results/admissibility.json`.

Three reasons to distrust δ, stated together. First, our two estimates disagree by a factor of 1.4 and the headline holds only under the smaller. Second, even within task 0 the classes overlap — the largest success sits at 1.45 cm and the smallest failure at 1.41 cm — so there is no clean threshold at n=10, only a rank association.

Third, and most serious: both estimates are measured on the *container* during the place phase, and Table 4 applies the result to the *grocery* during the grasp phase. A parallel-jaw grasp tolerance on a 6 cm can and a release tolerance into an open basket are not the same physical quantity, and we have not measured the first at all.

The experiment that would fix all three is the same one: sweep a deliberate displacement of the grasp constant, per object, and read the radius off the curve. It is implemented (`--goal-anchor-jitter-cm`) and unrun.

D Sensitivity of H_δ to the tolerance

The quantisation δ is a modelling choice, so the finding must not depend on it. Recomputing placement entropy across five tolerances spanning two orders of magnitude:

suite	0.5mm	2mm	5mm	1cm	5cm
Object	**2.11**	**0.48**	**0.00**	**0.00**	0.00
Spatial	5.48	4.49	2.04	0.25	0.00
Goal	5.59	5.08	1.53	0.02	0.00
Long	5.63	5.40	4.27	0.37	0.00

Table 18: Mean placement entropy H_δ in bits, swept over an order of magnitude in the tolerance δ. LIBERO-Object is lowest at every tolerance, so the ordering does not depend on the modelling choice — but every suite reaches zero by $\delta = 5\,\text{cm}$, which is a statement about coarse resolution rather than about LIBERO. That degeneracy is why the paper's claim is licensed on the radius R (Table 4), which has no such parameter. Artifact: `results/placement_entropy.json`.

The ordering is unchanged wherever the measure discriminates at all. Coarsening δ lowers every suite's entropy, as it must — coarser radii merge components — and by $\delta = 5\,\text{cm}$ every suite reaches zero, which is a statement about coarse resolution rather than about LIBERO. This is precisely why the paper's claim is licensed on the radius R (Table 4), which has no such parameter: at $\delta = 5\,\text{cm}$ every suite would look admissible on H, while on D only LIBERO-Object is, at any tolerance the controller actually has. Artifact: `results/placement_entropy.json`.

References

[1] Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *NeurIPS*, 2019.

[2] Senyu Fei et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.

[3] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel,

Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.

[4] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT*, 2018.

[5] Tianchong Jiang, Xiangshan Tan, Samuel Wheeler, Luzhe Sun, Tewodros W. Ayalew, and Matthew Walter. What are we actually benchmarking in robot manipulation? *arXiv preprint arXiv:2606.04233*, 2026.

[6] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[7] Ajay Mandlekar et al. What matters in learning from offline human demonstrations for robot manipulation. In *CoRL*, 2021.

[8] R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2019.

[9] Wilbert Pumacay, Ishika Singh, Jiafei Duan, Ranjay Krishna, Jesse Thomason, and Dieter Fox. THE COLOSSEUM: A benchmark for evaluating generalization for robotic manipulation. In *Robotics: Science and Systems (RSS)*, 2024.

[10] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do imagenet classifiers generalize to imagenet? In *International Conference on Machine Learning (ICML)*, 2019.

[11] Jesse Thomason, Daniel Gordon, and Yonatan Bisk. Shifting the baseline: Single modality performance on visual navigation and QA. In *NAACL*, 2019.

[12] Antonio Torralba and Alexei A. Efros. Unbiased look at dataset bias. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011.

[13] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.